

TripMate AI: Intelligent Group Activity Planning Platform

Powered by a Unified Deep-Reasoning Agent

Project Idea, Features & Technical Architecture Proposal for Supervisor

Project Team (6 Members): Ta Quang Huy Ha Dang Huy Pham Dinh Anh Duong Nguyen Minh Duc
 Nguyen Tung Duong Dinh Tien Luan

1. Introduction & Project Idea

1.1. The Problem

Organizing group activities — ranging from a half-day city hangout (cafe hopping, casual dining, entertainment) to a multi-day self-guided vacation — is painful in practice:

- **Manual research** across maps, social reviews and booking sites takes hours.
- **Schedule overlap** across members with different free windows ($\pm\delta$ tolerance) is a combinatorial puzzle.
- **Conflicting preferences** (chill cafe vs. lively pub, budget street food vs. upscale dining, nature vs. city spots) lead to decision paralysis.
- **Fragmented chat** on Zalo/Messenger buries plans, locations and links.
- **One organizer** carries the entire load and gets blamed if the group is unhappy.
- **Expense splitting** after the outing or trip is tedious and awkward.

1.2. The Solution

TripMate AI turns group planning into a guided, consensus-driven flow in under 15 minutes, supporting both **City Hangouts** (single-day cafe, dining, and entertainment routes) and **Multi-Day Travel Trips** (intercity getaways, homestays, and vacations): members chat freely in a shared room → an AI agent asks clarifying questions → the agent researches and proposes a tailored itinerary → the group votes leg-by-leg → the plan is locked and expenses are auto-settled.



Figure 1: End-to-end collaborative planning workflow

2. Platform Features

- **Dual Planning Scopes with Shared Micro-Granularity:**
 - *City Hangouts & Day Outings:* Rapid curation of micro-spots (specialty cafes, local eateries, cinemas, workshops, chill spots) with real-time traffic avoidance.
 - *Multi-Day Travel Itineraries:* Macro-planning across multiple days (homestays/hotels, intercity transit), where **each individual day is planned with the exact same high-resolution micro-detail** (specific cafes, lunch spots, afternoon activities, and evening hangouts).
- **Group Room:** Shared chat room, invite by link. Members chat freely — budget, dietary needs, dates and vibe preferences are shared naturally in conversation or collected by the agent when it detects they are needed. 6 event types: TRAVEL, DINING, HANGOUT, ENTERTAINMENT, SIGHTSEEING, CUSTOM. Roles: Host, Member, Viewer.
- **AI Planning Hub:** One-click itinerary generation, curated cafe & dining suggestions, local activity/lodging lookup, and smart sequence optimization for self-guided groups.
- **Voting & Customization:** Leg-by-leg UP/DOWN voting with instant alternatives, live AI chat to tweak the plan, manual drag-and-drop editor.

- **Map & Utilities:** Interactive map (Mapbox), weather-based packing checklist, PDF export, Vietnamese/English UI.
- **Expense Splitting:** Tracks advances and payments, supports equal/exact/percentage splits, computes minimum peer-to-peer settlements.

3. The AI Agent: Tools, Skills & Operational Flow

3.1. Model Layer

The agent is powered by **DeepSeek-V4**, utilizing two operational tiers within a unified context:

- **Flash** — fast chat interaction, room message parsing, and tool-call formatting.
- **Pro** — deep reasoning for constraint solving, spatial planning, and preference arbitration.

3.2. Tools (External Capabilities)

Tools are typed callable functions providing real-time data for both local outings and multi-day travel:

Tool	Function & Parameters → Output	Provider
<code>search_places</code>	query, location, radius → cafes, eateries, chill spots, POIs, ratings, hours	Google Places
<code>search_activities</code>	location, category → entertainment, workshops, local experiences	Klook
<code>search_hotels</code>	location, dates, guests → homestays, hotels & lodging rates	Booking.com
<code>search_transit</code>	origin, destination, dates, pax → flight / train / intercity transit options	Amadeus / Transit
<code>get_weather</code>	location, date range → forecast & weather alerts	OpenWeatherMap
<code>get_route</code>	origin, destination, mode, departure_time → duration, distance	Mapbox

Execution policy: Independent tool calls (e.g., weather + lodging + spots) run **in parallel**; dependent calls (e.g., routing between chosen spots) execute sequentially. All calls are logged in `agent_logs`.

3.3. Agent Skills

Internal capabilities governing agent behavior beyond raw API querying:

- **Research skill:** contextual query formulation, multi-source retrieval, and candidate ranking (Section 3.4).
- **Clarification skill:** extracts stated preferences from chat (budget, dietary rules, vibe/culinary tastes, transport mode); only queries missing critical parameters without repetitive intake forms.
- **Conflict-mediation skill:** negotiates compromises for conflicting preferences (e.g., chill cafe vs. energetic workshop) through hybrid options.
- **Local-repair skill:** patches downvoted itinerary legs with pre-scored alternatives without full re-planning.
- **Validation skill:** verifies feasibility (prices > 0, dates within window, valid opening hours) before presenting options.

3.4. Research Skill: Finding Places, Cafes & Restaurants

The Research skill transforms group preferences into vetted, ranked candidates through a streamlined process:

1. **Contextual Querying:** Embeds dietary needs, group size, and location proximity into queries (e.g., `search_places("spacious cafe with parking and power outlets", near=MeetingPoint party=6)`).
2. **Multi-Source Retrieval:** Queries relevant APIs (Google Places for cafes/dining/POIs, Klook for local experiences, Booking.com for homestays/hotels).

3. **Hard Filtering:** Eliminates candidates exceeding group budget cap, violating dietary rules, lacking party capacity, or closed during target visit slots.
4. **Heuristic Scoring:** Evaluates each surviving candidate using a multi-factor score:

$$\text{Score}(c) = w_1 \cdot \text{Rating}(c) + w_2 \cdot \text{PriceFit}(c) + w_3 \cdot \text{Proximity}(c) + w_4 \cdot \text{PreferenceMatch}(c)$$

Plain-Language Explanation of Terms:

- $\text{Rating}(c)$: Platform rating and quality score from real user reviews (0–1).
 - $\text{PriceFit}(c)$: Affordability relative to group budget per person (higher score for budget-friendly spots; penalized if exceeding cap).
 - $\text{Proximity}(c)$: Closeness to the current cluster or anchor point (e.g., hotel, meeting point), minimizing transit time.
 - $\text{PreferenceMatch}(c)$: How well the venue matches stated vibes and tastes (e.g., quiet work cafe, outdoor rooftop, vegan dish) minus penalties if similar styles were previously rejected.
 - *Dynamic Weights* (w_1 – w_4): Importance sliders that auto-tune based on chat (e.g., mentioning “cheap eats” boosts w_2 ; wanting “somewhere nearby” boosts w_3).
5. **Selection & Standbys:** The top candidate forms the proposed leg, while 2–3 next-best options are retained as instant voting alternatives.

3.5. Destination Discovery: Unset Destinations & Outing Themes

When the destination or plan is unspecified (e.g., “*somewhere cool for weekend getaway, 2–3 days, \$100/pax*” or “*a fun Saturday city tour with cafes and boardgames near district center*”):

- **Stage A (Candidate Generation):** LLM analyzes constraints and intent (multi-day travel → shortlist destinations like Tam Dao, Ninh Binh, Cat Ba, Da Lat; city hangout → cluster themes like Old Quarter cafe hop, West Lake sunset dining, or entertainment hub).
- **Stage B (Live Validation):** Parallel tool checks verify weather (`get_weather`), transit feasibility (`search_transit` or `get_route`), and spot density (`search_places`). Surviving top 2–3 concepts are presented to the group for a preliminary vote.

3.6. Place Selection Reasoning: Spatial & Temporal Coherence

Individual scores do not guarantee a smooth schedule. Whether organizing a standalone 1-day city tour or Day 3 of a vacation, the agent sequences places into logical, high-detail daily blocks:

- **Geographic Clustering:** Groups candidate spots (morning coffee → lunch specialty dining → afternoon activity → evening hangout) into spatial clusters relative to an anchor (e.g., hotel or meeting spot). Choosing places within the same cluster saves hours of transit.
- **Half-Day Time Blocks:** Partitions the day into Morning, Afternoon, and Evening blocks. Stops within each block are strictly co-located in the same neighborhood.
- **Anti-Zigzag Optimization:** Analyzes route geometry and applies nearest-neighbor sequencing to flexible stops, preventing backtracking across town.

3.7. Spatial Planning: Routing, Distance, Time & Transport

Spatial planning validates that the itinerary is physically feasible within group time budgets:

1. **Realistic Transit & Timing:** Computes real road distance and travel time via `get_route` with departure timestamps. Peak-hour transitions (7–9 am, 5–7 pm) automatically include a 30–50% traffic buffer.
2. **Feasibility Checks:** Validates time windows for every stop using the condition:

$$\text{Arrival Time} + \text{Visit Duration} + \text{Slack} \leq \text{Closing Time}$$

Plain-Language Explanation: Ensures the group reaches the venue, enjoys it for the planned duration (e.g., 60 min for a cafe, 90 min for lunch), and departs before closing time, while maintaining a safety buffer ($\text{Slack} \geq 15 \text{ min}$) for unexpected traffic. Fixed meal slots (lunch 11:30–13:30, dinner 18:00–20:30) are preserved.

3. Transport Mode Selection: Chooses optimal transit based on group size, distance, and context:

Distance / Context	Default Mode	Rationale
< 1.5 km	Walking	Zero cost, walkable in dense areas, no waiting time.
2–15 km (flexible / youth)	Motorbike / Scooter	Highly agile in traffic, easy parking, cost-effective, scenic exploration.
2–10 km (groups / rain)	Grab / Taxi	Weather-safe, convenient point-to-point transit for small groups.
10–40 km (≥ 5 pax)	Private Van / Car	Keeps group together, lower cost than multi-taxi convoy.
> 40 km	Intercity Bus / Flight	Optimizes transit duration versus budget limits.

Override: Modes dynamically adjust if the group specifies a preference (e.g., “rent motorbikes”) or during adverse weather.

Example Spatial Optimization (Da Nang / Local Day Trip, 6 pax): Rather than scattering far-apart spots in the same morning, the agent anchors the morning activity and on-site lunch together, returns off-peak at 14:00, and assigns nearby seaside cafes and sunset hangouts for the evening. Total daily transit is reduced from 180 min to 90 min.

3.8. Agentic Reasoning Loop

TripMate AI runs an adaptive **Think** → **Act** → **Observe** loop rather than a rigid pipeline:

1. **Think:** Evaluates the structured session state (Section 3.9) to choose the single next best action (call a tool, query the group, or emit proposal) using DeepSeek-Pro.
2. **Act:** Executes the selected action (e.g., invoke `search_places` or Clarification skill).
3. **Observe:** Parses output, updates state, and evaluates stop conditions.
4. **Stop Condition:** Terminates when the itinerary meets all constraints, consensus is reached, or round limits (Section 3.10) are met.

The 5 UX stages in Figure 1 provide user-facing checkpoints, while internal execution remains dynamic and dependency-driven.

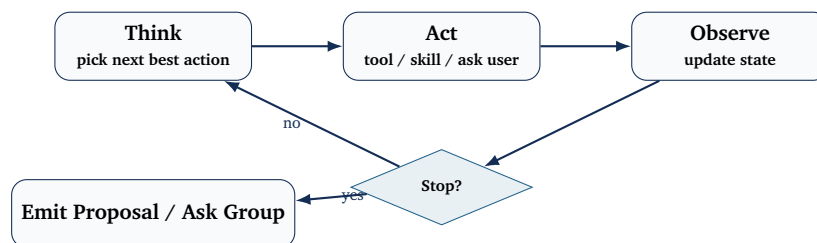


Figure 2: Internal Think-Act-Observe loop: the agent dynamically determines actions each turn until constraints and consensus are satisfied (Section 3.10)

3.9. Context Management

- Maintains a running **structured JSON state** (extracted constraints, budget, booked legs) updated each turn, avoiding bloated token re-transmission.
- The itinerary is maintained as a stateful object of uniquely ID'd legs, enabling atomic local repairs.

3.10. Retry & Error Handling

- External API calls allow up to **2 retries** with exponential backoff before falling back to cached data (Redis, TTL 6–24h) with an “outdated” indicator.
- Unresolvable tool failures are stated transparently to avoid hallucinations.
- Clarification interactions are capped at **3 rounds**; unaddressed inputs default to sensible heuristics.

3.11. Learning Feedback Loop

- Rejection reasons (price, distance, style, vibe) are logged in the session state.

- Subsequent candidate proposals filter out attributes matching rejected patterns.
- Feedback is strictly session-scoped, resetting for subsequent trips.

4. Backend Architecture

TripMate AI adopts an asynchronous, modular backend in Python 3.11+, unifying RESTful services and AI reasoning pipelines within a single codebase:

- **Framework & API Layer:** Built with **FastAPI** (ASGI/Uvicorn) for high-throughput asynchronous I/O, automatic OpenAPI 3.1 documentation ('/docs'), and native **Pydantic v2** request/response validation. Routers are modularly segmented (/api/v1/auth, /events, /plans, /votes, /places, /expenses, /ai).
- **Database & ORM Layer:** **PostgreSQL** paired with **SQLAlchemy 2.0 (Async)** and **Alembic** migrations. The schema models core entities (User, Event, EventMember, Plan, PlanStop, PlanVote, ChatMessage, Expense, Settlement) with JSONB fields for flexible AI metadata and pgvector indexing.
- **Real-Time & Caching Layer:**
 - **Redis (Async):** Multi-tier caching for external APIs (Google Places TTL 6–24h, Weather TTL 1–3h), session store, and sliding-window rate limiting.
 - **WebSockets / SSE:** Powers bi-directional room chat, live voting status broadcasts, and low-latency token streaming for AI reasoning outputs.
- **Background Processing:** **Celery / FastAPI BackgroundTasks** handles long-running asynchronous jobs including PDF itinerary compilation (WeasyPrint) and email notifications (aiosmtplib).
- **Expense Settlement Engine:** Built on 3 core tables (expenses, expense_splits, settlements). Calculates net balances:

$$\text{Net}_u = \sum \text{Paid}_u - \sum \text{Owed}_u$$

Plain-Language Explanation: For each member u , Net_u is total money paid upfront minus their fair share. Positive means the group owes them money; negative means they owe the group. A greedy matching algorithm pairs the largest debtor with the largest creditor iteratively, settling all debts in the minimum number of direct transfers (N members require at most $N - 1$ transactions).

5. Frontend Architecture

The frontend is structured as a reactive, type-safe Single Page Application (SPA) designed for responsive real-time collaboration:

- **Core Stack:** **React 18 + Vite + TypeScript** with Strict Mode, ensuring sub-second HMR during development and full type synchronization with backend Pydantic schemas.
- **UI System & Accessibility:** **Tailwind CSS + shadcn/ui** (Radix UI primitives), enforcing WCAG AA contrast standards, mobile-first responsiveness, and cohesive design tokens across all views.
- **State Management & Data Syncing:**
 - **Zustand:** Lightweight global store for client-side authentication, active room session, and UI drawer states.
 - **TanStack Query v5:** Handles server-state caching, automatic background re-fetching, request deduplication, and optimistic updates for instant voting and expense feedback.
 - **React Hook Form + Zod:** Provides high-performance, client-side form validation mirroring API contracts.
- **Key Feature Modules:**
 - **Real-Time AI Workspace:** Live SSE chat streaming interface, interactive Proposal Cards with instant-swap buttons, and dynamic prompt-chip suggestions.
 - **Visual Plan & Map Canvas:** **Mapbox GL JS** integration rendering geocoded stop pins, route polylines, and drag-and-drop stop reordering.

- **Consensus & Voting Dashboard:** Real-time UP/DOWN voting progress bars, leg-level discussion threads, and compromise suggestion previews.
- **Financials & Utilities:** Multi-mode bill split calculator (equal/custom/ratio), settlement QR/bank transfer cards, weather-adaptive packing checklist, and bilingual support (**react-i18next** for VI/EN).

6. Deployment & Security

- **Pipeline:** Docker Compose (local/staging), Vercel (frontend CDN), Render/VPS (backend), GitHub Actions (CI).
- **Security:** bcrypt password hashing, JWT access + Redis-rotated refresh tokens, RBAC, **bleach** sanitization, Pydantic v2 schema validation on all AI output.
- **Data privacy:** all trip-specific information (dietary needs, budget, preferences) lives only in the session chat and the agent's session state; it is not stored in permanent user profiles. Access is scoped by RBAC and not shared outside the room.
- **Rate limiting & audit:** Redis-backed limits on external and AI endpoints; all tool calls and token usage logged in `agent_logs`.

7. Expected Impact

- **Speed:** itinerary consensus in under 15 minutes instead of days of back-and-forth.
- **Fairness:** mathematically resolved schedule/preference conflicts, no single organizer carrying the load.
- **Reliability:** explicit retry, fallback and validation rules keep the agent from silently failing or hallucinating.

TripMate AI turns the chaos of group planning into a seamless, collaborative, and intelligent experience.